

Template e declaração de variáveis

```
#include <bits/stdc++.h>
#define f first
#define s second
#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define dbg(x) cout << #x << " = " << x << endl
using namespace std;

long long linha, coluna;
```

Declaração das variáveis lin e col como globais pois elas são usadas em todas as funções.

MAIN

```
int main(){
    _;

    cin >> linha >> coluna;

    vector<vector<int>> matriz(linha, vector<int>(coluna));

    for(int i = 0; i < linha; i++){
        for(int j = 0; j < coluna; j++){
            cin >> matriz[i][j];
        }
    }

    cout << solve(matriz)<< endl;

    return 0;
}
```

Na main ocorre a leitura da matriz binária, que nesse caso é um vetor de vetor de inteiro, e após isso a função solve é chamada e recebe como parâmetro a matriz.

SOLVE

```
long long solve(vector<vector<int>> &matriz){
    vector<long long> altura(coluna, 0);

    long long resposta = 0;

    for(int i = 0; i < linha; i++){
        for(int j = 0; j < coluna; j++){
            if (matriz[i][j] == 1) altura[j]++;
            else altura[j] = 0;
        }

        resposta = max(resposta, AreaRetangulo(altura));
    }

    return resposta;
}
```

Na função solve, é criado um vetor chamado altura para armazenar quantos números 1 consecutivos existem na coluna. Para isso, a matriz é percorrida novamente e quando a célula atual é o número 1, o vetor altura no índice j é incrementado em uma unidade. Caso contrário, o vetor que armazena as alturas é zerado no índice j. Após percorrer cada linha, é comparado, a variável “resposta” com com o valor retornado pela função AreaRetangulo(altura). Essa função recebe o vetor de alturas construído até a linha atual e calcula a maior área retangular possível. A variável resposta armazena a maior área encontrada entre todas as linhas da matriz

AREARETANGULO

```
long long AreaRetangulo(vector<long long> &altura){  
    long long resposta = 0;  
    stack <int> s;  
    for(int i = 0; i < altura.size(); i++){  
        while(s.size() != 0 && altura[s.top()] >= altura[i]){  
            long long topo = s.top();  
            s.pop();  
            long long largura;  
            if(s.size() == 0) largura = i;  
            else largura = i - s.top() - 1;  
            resposta = max (resposta, altura[topo] * largura);  
        }  
        s.push(i);  
    }  
    while (s.size()!=0) {  
        int topo = s.top();  
        s.pop();  
        int largura;  
        if (s.empty()) largura = altura.size();  
        else largura = altura.size() - s.top() - 1;  
        resposta = max(resposta, altura[topo] * largura);  
    }  
    return resposta;  
}
```

O objetivo dessa função é encontrar a maior área retangular possível utilizando uma pilha. A pilha armazena os índices das colunas e é mantida de forma que as alturas correspondentes a esses índices estejam em ordem crescente. Caso a altura da coluna atual seja menor ou igual ao valor da coluna que está no topo da pilha, o índice do topo é removido e será usado como altura para calcular a área do retângulo. A largura é calculada a partir utilizando o índice da coluna atual e o índice da coluna que

ficou no topo. Caso a pilha esteja vazia durante o cálculo da largura, esta terá como valor o índice atual, uma vez que não existe nenhuma coluna menor que essa à esquerda. Após percorrer todas as colunas, se ainda existir algum valor na pilha, o mesmo processo será feito novamente. A única mudança é que caso a pilha esteja vazia, a largura será igual ao número de colunas indicando que o retângulo pode ocupar toda a extensão do histograma.